MokA: Forward/Backward Math From Code

Derived directly from repository implementation

February 24, 2026

1 Scope and Grounding

This note is derived from these files:

- `models/unified_arch.py`: multimodal embedding and modality masks.
- `models/modeling_llama.py`: masked LoRA projections used by attention/MLP and LM loss.
- `peft_hyper/tuners/lora.py`: custom `Linear` with `lora_A0/A1/A2` and shared `lora_B0`.
- `scripts/finetune/finetune.py`: which modules are wrapped and which parameters are trainable.
- `trainer.py`: gradient-sensitivity metric definitions.

2 Step-by-Step in Plain English (First)

This section explains the whole forward/backward pipeline without equations first.

2.1 Big picture

Each adapted linear layer in MokA does two things and adds them:

- Base path: normal frozen pretrained linear transform.
- LoRA path: small trainable correction built from text/video/audio branches.

Final output of the layer is:

$$\text{base output} + \text{LoRA correction}.$$

2.2 What happens before LoRA

1. Text tokens are embedded by the LLM embedding table.
2. Video/audio raw features are encoded and then projected into the same LLM hidden space (4096-dim).
3. These modality features are inserted into the token sequence.
4. Binary masks are built to mark which positions are text, video, audio, and question tokens.

2.3 Inside one LoRA-adapted linear layer

1. Compute normal base linear output.
2. Split the same input into three masked streams: text-only, video-only, audio-only.
3. Pass each stream through its own low-rank A matrix: A_0 (text), A_1 (video), A_2 (audio).
4. Improve video and audio streams using question-conditioned cross-attention: video/audio query attends to question-text keys/values.
5. Add that correction back with scale `blc_weight` (called β in math).
6. Sum the three branch features and pass once through shared B_0.
7. Add this LoRA output to base output.

2.4 Why "A + cross-attention + B"?

- A_2 (audio branch) alone gives unimodal audio adaptation.
- Cross-attention injects question/text context into that audio branch.
- Shared B_0 maps fused low-rank branch features back to layer output dimension.

So yes: first A branch features, then cross-modal conditioning, then shared B projection.

2.5 Backward pass in simple words

1. CE loss gives gradient on logits.
2. Gradients flow back through decoder layers.
3. For each adapted layer, gradients split into: base path and LoRA path.
4. In LoRA path, gradients reach: B_0, then branch features, then $A_0/A_1/A_2$.
5. A_1 and A_2 get gradients from both direct residual path and cross-attention path.
6. Optimizer updates only parameters still marked `requires_grad=True`.

2.6 Minimal glossary

- B: batch size (your run: 4).
- T: sequence length after multimodal insertion and padding (dynamic).
- d: hidden dimension (for Llama-2-7B, $d = 4096$).
- Mask M: binary selector (1 keep token, 0 zero token for that branch).
- Broadcasting: mask shape $[B, T, 1]$ is automatically repeated across hidden dim to match $[B, T, d]$ for elementwise multiplication.
- U: branch low-rank features after A projection (shape $[B, T, r]$).
- O: cross-attention output used to refine video/audio branch features.
- β: same as `blc_weight`; scales cross-attention correction.

3 Math Mapping (Second)

Now the same story with equations.

3.1 Simple Overall Math (Read This First)

If you only remember one block, remember this:

1) Build masked inputs

$$X_t = X \odot M_t, \qquad X_v = X \odot M_v, \qquad X_a = X \odot M_a.$$

2) Get three LoRA branch features

$$H_t = sA_0(X_t), \qquad H_v = sA_1(X_v), \qquad H_a = sA_2(X_a).$$

3) Inject question context into video/audio

$$\widehat{H}_v = H_v + \beta\,(M_v \odot \mathrm{Attn}(H_v,\ H_t \odot M_q,\ H_t \odot M_q)),$$

$$\widehat{H}_a = H_a + \beta\,(M_a \odot \mathrm{Attn}(H_a,\ H_t \odot M_q,\ H_t \odot M_q)).$$

4) Fuse branches and project by shared B

$$H_{\text{fuse}} = H_t + \widehat{H}_v + \widehat{H}_a, \qquad Y_{\text{lora}} = B_0(H_{\text{fuse}}).$$

5) Add LoRA correction to frozen base linear

$$Y = XW_0^\top + b + Y_{\text{lora}}.$$

6) Backward summary Loss gradient flows:

$$\frac{\partial L}{\partial Y} \to \frac{\partial L}{\partial B_0},\ \frac{\partial L}{\partial A_0},\ \frac{\partial L}{\partial A_1},\ \frac{\partial L}{\partial A_2}.$$

So "what is happening" is: masked modality features → low-rank branch transforms → question-conditioned fusion → shared projection → add to base layer.

3.2 Shape-Annotated Version

This is the same pipeline with shape tags as subscripts.

$$X_{[B\times T\times d_{\text{in}}]},\ M_t, M_v, M_a, M_{q[B\times T\times 1]},\ A^a, A^v, A^t{}_{[r\times d_{\text{in}}]},\ B_{[d_{\text{out}}\times r]},\ W_{0[d_{\text{out}}\times d_{\text{in}}]}.$$

Masked inputs

$$X_{t[B\times T\times d_{\text{in}}]} = X_{[B\times T\times d_{\text{in}}]} \odot M_{t[B\times T\times 1]},$$

$$X_{v[B\times T\times d_{\text{in}}]} = X \odot M_v, \qquad X_{a[B\times T\times d_{\text{in}}]} = X \odot M_a.$$

Unimodal A projections

$$H_{t[B\times T\times r]} = sA^t X_t, \qquad H_{v[B\times T\times r]} = sA^v X_v, \qquad H_{a[B\times T\times r]} = sA^a X_a.$$

Cross-attention and enhancement

$$C_{v[B\times T\times r]} = \mathrm{Attn}(H_v,\ H_t \odot M_q,\ H_t \odot M_q),$$

$$C_{a[B\times T\times r]} = \mathrm{Attn}(H_a,\ H_t \odot M_q,\ H_t \odot M_q),$$

$$\widehat{H}_{v[B\times T\times r]} = H_v + \beta(M_v \odot C_v), \qquad \widehat{H}_{a[B\times T\times r]} = H_a + \beta(M_a \odot C_a).$$

Fusion and shared B

$$H_{\mathrm{fuse}[B\times T\times r]} = H_t + \widehat{H}_v + \widehat{H}_a,$$

$$Y_{\mathrm{lora}[B\times T\times d_{\mathrm{out}}]} = B(H_{\mathrm{fuse}}).$$

Final output

$$Y_{[B\times T\times d_{\mathrm{out}}]} = XW_0^\top + b + Y_{\mathrm{lora}}.$$

Backward key shapes

$$\Delta_{[B\times T\times d_{\mathrm{out}}]} = \frac{\partial L}{\partial Y_{\mathrm{lora}}}, \qquad \left(\frac{\partial L}{\partial H_{\mathrm{fuse}}}\right)_{[B\times T\times r]} = \Delta B,$$

$$\left(\frac{\partial L}{\partial B}\right)_{[d_{\mathrm{out}}\times r]}, \qquad \left(\frac{\partial L}{\partial A^t}\right)_{[r\times d_{\mathrm{in}}]}, \left(\frac{\partial L}{\partial A^v}\right)_{[r\times d_{\mathrm{in}}]}, \left(\frac{\partial L}{\partial A^a}\right)_{[r\times d_{\mathrm{in}}]}.$$

3.3 MokA Paper Notation (Aligned to Eq. 4–13)

The paper writes modality tokens as:

$$x = [x^a; x^v; x^t]$$

(audio, visual, text). Modality-specific low-rank A matrices:

$$A^a,\ A^v,\ A^t,$$

and a shared multimodal B matrix:

$$B.$$

Unimodal compression (paper Eq. 4).

$$Ax = [A^a x^a;\ A^v x^v;\ A^t x^t].$$

Cross-attention enhancement (paper Eq. 5–9).

$$Att_{a,t,t} = Att(A^a x^a,\ A^t x^t,\ A^t x^t),$$

$$Att_{v,t,t} = Att(A^v x^v,\ A^t x^t,\ A^t x^t),$$

$$\widetilde{A^a x^a} = A^a x^a + \lambda_a Att_{a,t,t},$$

$$\widetilde{A^v x^v} = A^v x^v + \lambda_v Att_{v,t,t},$$

$$\widetilde{Ax} = [\widetilde{A^a x^a};\ \widetilde{A^v x^v};\ A^t x^t].$$

Shared projection (paper Eq. 10).

$$BAx = [B\widetilde{A^a x^a};\ B\widetilde{A^v x^v};\ BA^t x^t].$$

Full adapted forward (paper Eq. 11–13 style).

$$h = W_0 x + \Delta W x,$$
$$h = W_0 x + [B(A^a x^a + \lambda_a Att_{a,t,t});\ B(A^v x^v + \lambda_v Att_{v,t,t});\ BA^t x^t],$$
$$h = W_0 x + \underbrace{[BA^a x^a;\ BA^v x^v;\ BA^t x^t]}_{\text{unimodal adaptation}} + \underbrace{[\lambda_a BAtt_{a,t,t};\ \lambda_v BAtt_{v,t,t};\ 0]}_{\text{cross-modal adaptation}}.$$

Code mapping to paper symbols.

- paper $A^a, A^v, A^t \leftrightarrow$ code `lora_A2`, `lora_A1`, `lora_A0` (audio/visual/text).
- paper shared $B \leftrightarrow$ code `lora_B0`.
- paper $\lambda_a, \lambda_v \leftrightarrow$ code $\beta =$ `blc_weight` (single shared value in this implementation).

3.4 Detailed Math (After the Overview)

4 Notation and Shapes

Per adapted linear layer:

- Input: $X \in \mathbb{R}^{B \times T \times d_{\text{in}}}$.
- Base weight: $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, bias b.
- LoRA branch matrices:
$$A_0 \in \mathbb{R}^{r \times d_{\text{in}}}, \quad A_1 \in \mathbb{R}^{r \times d_{\text{in}}}, \quad A_2 \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B_0 \in \mathbb{R}^{d_{\text{out}} \times r}.$$
- Scale $s = \alpha / r$ (code uses `self.scaling[0]` for all branches).
- Binary masks from input construction:
$$M_t, M_v, M_a, M_q \in \{0, 1\}^{B \times T \times 1}.$$

4.1 Concrete values in your current AVE run

From `scripts/finetune/ft_ave.sh` and `scripts/finetune/finetune.py`:

- $B = 4$ per GPU (`--per_device_train_batch_size 4`), and with 1 GPU in your sbatch run, effective micro-batch is 4.
- LLaMA hidden size is $d_{\text{model}} = 4096$.
- LoRA rank setting is `lora_r=444`. In this custom code, that is parsed digit-wise, so
$$r_0 = r_1 = r_2 = 4, \quad d_k = 4.$$
- LoRA scale uses $s = \alpha / r$ with `lora_alpha=16`, so $s = 16/4 = 4$.
- β in this document is `blc_weight`; in your script `--blc_weight 1`, so $\beta = 1$.
- T is dynamic per batch after multimodal insertion and left-padding; it is *not* a fixed constant in code.

About T in practice. For each sample, final sequence length is:

$$T_i = n^{(i)}_{\text{text-kept}} + n^{(i)}_{\text{video-feat}} + n^{(i)}_{\text{audio-feat}},$$

then batch $T = \max_i T_i$ after left-padding. In your setup, projector query counts are 32 for visual and 32 for audio, so inserted visual/audio feature-token counts are controlled by those projector outputs.

Important implementation constraint. In code, `r` is parsed digit-wise (`str(r)`), and branch outputs are later summed. Therefore branch ranks must match in practice (or the sum would fail).

5 Core Operators (Plain Definitions)

What does $X_t = X \odot M_t$ mean? $\odot$ is elementwise product with broadcast on the last dimension:

$$X_t[b, t, :] = X[b, t, :] \cdot M_t[b, t, 1].$$

So if $M_t[b, t, 1] = 1$, token (b, t) is kept; if 0, that token is zeroed for the text branch. Same rule for X_v, X_a.

What is a linear map? For one token vector $x \in \mathbb{R}^{d_{\text{in}}}$ and weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

$$\text{Linear}(x; W, b) = xW^\top + b.$$

Applied to all (b, t) positions, this is exactly PyTorch `nn.Linear`.

What is β? β is the scalar multiplying cross-modal correction in video/audio branches:

$$\widetilde{U}_1 = U_1 + \beta(M_v \odot O_v), \quad \widetilde{U}_2 = U_2 + \beta(M_a \odot O_a).$$

In your run, $\beta = 1$.

6 Multimodal Input Construction

The model builds one embedding sequence and four aligned masks:

$$E \in \mathbb{R}^{B \times T \times d_{\text{model}}}, \quad M_t, M_v, M_a, M_q \in \{0, 1\}^{B \times T \times 1}.$$

Text tokens map through token embedding; image/video/audio special tokens are replaced by projector features; labels for non-text inserted modal features are set to -100 (ignored in CE). Then sequences are left-padded to common T.

6.1 Math for unimodal compression (projectors)

Let raw visual encoder output be V_{raw} and raw audio encoder output be A_{raw}. Projectors (`vl_projector`, `al_projector`) produce language-space tokens:

$$V_{\text{proj}} = f_{\text{vl}}(V_{\text{raw}}) \in \mathbb{R}^{n_v \times 4096}, \qquad A_{\text{proj}} = f_{\text{al}}(A_{\text{raw}}) \in \mathbb{R}^{n_a \times 4096}.$$

These are "unimodal compression" maps from modality-native features to LLM token space. They happen *before* LoRA inside the decoder: first you build E, then decoder layers consume E.

7 Compact Derivation (No Repetition)

The previous sections already gave the full forward recipe. This section keeps only the minimum math you need for understanding gradients.

Single-line forward for one adapted layer.

$$Y = XW_0^\top + b + B_0(H_{\text{fuse}}).$$

Here H_{fuse} is the fused low-rank feature from the three modality branches (paper-aligned form given above).

What "B multiplies A" means (core idea). Ignoring cross-attention details for one branch:

$$B_0(sA_iX_i) = X_i(sB_0A_i)^\top,$$

so effective low-rank update is:

$$\Delta W_i = sB_0A_i, \qquad W_{\text{eff}} = W_0 + \sum_i \Delta W_i.$$

8 Where It Is Used

The wrapped linear layers are injected into

$$\{q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj\}$$

across decoder layers. In attention and MLP forward passes, modality masks are passed to these projections.

9 Language Modeling Loss

Given logits $Z \in \mathbb{R}^{B\times T\times V}$ and labels y:

$$Z_{\text{shift}} = Z_{:,1:T-1,:}, \quad y_{\text{shift}} = y_{:,2:T}.$$

Loss is cross-entropy over flattened tokens with ignore index -100:

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{n\in\mathcal{I}} \log \operatorname{softmax}(Z_n)_{y_n}.$$

10 Backward Propagation (Compact)

10.1 From CE to logits

For each valid token row n:

$$\frac{\partial \mathcal{L}}{\partial Z_n} = \frac{1}{N_{\text{valid}}} \left(\operatorname{softmax}(Z_n) - e_{y_n}\right),$$

and 0 for ignored rows.

10.2 LoRA gradients you care about

Let

$$\Delta = \frac{\partial \mathcal{L}}{\partial Y_{\text{lora}}}.$$

Then

$$\frac{\partial \mathcal{L}}{\partial B_0} = \sum_{b,t} \Delta_{b,t}^{\top} H_{\text{fuse},b,t}, \qquad \frac{\partial \mathcal{L}}{\partial H_{\text{fuse}}} = \Delta B_0.$$

For branch i with $H_i = sA_i(D(X_i))$:

$$\frac{\partial \mathcal{L}}{\partial A_i} = s \sum_{b,t} \Gamma_{i,b,t}^{\top} D(X_{i,b,t}),$$

where $X_i = X \odot M_i$ and $\Gamma_i = \partial \mathcal{L} / \partial H_i$.

In particular:

$$\boxed{\frac{\partial \mathcal{L}}{\partial A_1} = s \sum_{b,t} \Gamma_{v,b,t}^{\top} D(X_{v,b,t}), \qquad \frac{\partial \mathcal{L}}{\partial A_2} = s \sum_{b,t} \Gamma_{a,b,t}^{\top} D(X_{a,b,t})}$$

(Γ_v, Γ_a include both direct path and cross-attention path terms).

11 What Gets Updated

- Base wrapped weight is frozen in custom `Linear` (`weight.requires_grad=False`).
- `mark_only_lora_as_trainable` first disables non-LoRA params.
- Then finetune script applies final filter: a parameter is trainable iff its name contains one of `save_modules`.

So the *actual* update set is:

$$\Theta_{\text{update}} = \{\theta : \texttt{requires_grad}(\theta) = \texttt{True after final filter}\}.$$

12 Optimizer Update (conceptual)

For each $\theta \in \Theta_{\text{update}}$, AdamW-style step:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right).$$

13 Gradient Sensitivity Metrics in Trainer

For each group $g \in \{\text{A0,A1,A2,B0,projectors}\}$:

$$G_g^2 = \sum_{\theta \in g} \left\| \frac{\partial \mathcal{L}}{\partial \theta} \right\|_F^2, \quad P_g^2 = \sum_{\theta \in g} \|\theta\|_F^2,$$

$$\text{grad_norm}_g = \sqrt{G_g^2}, \quad \text{param_norm}_g = \sqrt{P_g^2}, \quad \text{relative_grad_norm}_g = \frac{\text{grad_norm}_g}{\text{param_norm}_g + \varepsilon}.$$

These are exactly what hooks/logging in `trainer.py` compute.

14 Code-to-Math Trace Checklist

For any adapted layer in your run:

1. Build X, M_t, M_v, M_a, M_q from packed multimodal input.
2. Compute base output $XW_0^\top + b$.
3. Compute three masked low-rank branch outputs U_0, U_1, U_2.
4. Fuse question-conditioned information into video/audio branches.
5. Sum branches, project by shared B_0, add to base.
6. Continue through attention/MLP/stack, then LM head and shifted CE.
7. Backprop: CE $\to$ logits $\to$ decoder $\to$ each adapted linear: obtain $\partial\mathcal{L}/\partial A_0, \partial\mathcal{L}/\partial A_1, \partial\mathcal{L}/\partial A_2, \partial\mathcal{L}/\partial B_0$.
8. Optimizer updates only parameters still marked trainable.

15 Appendix A: Exact Projection Dimensions (Llama-2-7B)

From your model config:

$$d_{\text{model}} = 4096, \quad d_{\text{ff}} = 11008, \quad n_{\text{heads}} = 32, \quad n_{\text{kv}} = 32, \quad d_{\text{head}} = 4096/32 = 128, \quad L = 32.$$

Module	d_{in}	d_{out}	Output shape
q_proj	4096	4096	$B \times T \times 4096$
k_proj	4096	4096	$B \times T \times 4096$
v_proj	4096	4096	$B \times T \times 4096$
o_proj	4096	4096	$B \times T \times 4096$
gate_proj	4096	11008	$B \times T \times 11008$
up_proj	4096	11008	$B \times T \times 11008$
down_proj	11008	4096	$B \times T \times 4096$

16 Appendix B: Exact LoRA Parameter Counts in Your Run

Your custom adapter uses:

$$\#\text{params per adapted linear} = 3rd_{\text{in}} + rd_{\text{out}}$$

because it has $A_0, A_1, A_2 \in \mathbb{R}^{r \times d_{\text{in}}}$ and one shared $B_0 \in \mathbb{R}^{d_{\text{out}} \times r}$. With $r = 4$:

$$\# = 12d_{\text{in}} + 4d_{\text{out}}.$$

Module type	Count per module	Numeric value
$d_{\text{in}} = 4096, d_{\text{out}} = 4096$	$3 \cdot 4 \cdot 4096 + 4 \cdot 4096$	65,536
$d_{\text{in}} = 4096, d_{\text{out}} = 11008$	$3 \cdot 4 \cdot 4096 + 4 \cdot 11008$	93,184
$d_{\text{in}} = 11008, d_{\text{out}} = 4096$	$3 \cdot 4 \cdot 11008 + 4 \cdot 4096$	148,480

Per decoder layer (7 adapted projections):

$$4 \times 65536 + 2 \times 93184 + 1 \times 148480 = 596{,}992.$$

Across 32 decoder layers:

$$596{,}992 \times 32 = 19{,}103{,}744 \text{ LoRA parameters.}$$

17 Appendix C: Gradient Tensor Count in Your Run

Per adapted projection, gradient tensors are:

$$\frac{\partial \mathcal{L}}{\partial A_0}, \frac{\partial \mathcal{L}}{\partial A_1}, \frac{\partial \mathcal{L}}{\partial A_2}, \frac{\partial \mathcal{L}}{\partial B_0} \Rightarrow 4 \text{ tensors.}$$

Per decoder layer: $7 \times 4 = 28$ tensors. Across 32 layers:

$$28 \times 32 = 896 \text{ LoRA gradient tensors per backward pass}$$

(plus projector and any other trainable groups selected by `save_modules`).

18 Appendix D: Before/After Summary

Before (input side). Raw modalities are encoded and compressed:

$$\text{video/image/audio raw} \to f_{\text{projector}}(\cdot) \to \text{LLM-space tokens.}$$

Tokens and modality masks are assembled, labels on inserted modality tokens set to -100.

Inside each adapted linear.

$$X \xrightarrow{\text{base } W_0} Y_{\text{base}}, \quad X \xrightarrow{\text{mask} + A_0/A_1/A_2 + \text{cross-attn} + B_0} Y_{\text{lora}},$$

$$Y = Y_{\text{base}} + Y_{\text{lora}}.$$

After (output side). Decoder stack $\to$ LM head $\to$ shifted CE loss $\to$ backprop to all trainable parameters.